Complexity Lower Bounds of Small Matrix Multiplication over Finite Fields via Backtracking and Substitution

Chengu Wang
wangchengu@gmail.com

Abstract

We introduce a new method for proving bilinear complexity lower bounds for matrix multiplication over finite fields. The approach combines the substitution method with a systematic backtracking search over linear restrictions on the first matrix A in the product $AB = C^T$. We enumerate restriction classes up to symmetry; for each class we either obtain a rank lower bound by classical arguments or branch further via the substitution method. The search is organized by dynamic programming on the restricted matrix A. As an application we prove that the bilinear complexity of multiplying two 3×3 matrices over $\mathbb{F}_2$ is at least 20, improving the longstanding lower bound of 19 (Bläser 2003). The proof is found automatically within 1.5 hours on a laptop and verified in seconds.

1 Previous Lower Bounds

Denote $\langle l, m, n\rangle$ the matrix multiplication tensor between an $l \times m$ matrix and an $m \times n$ matrix, and $\mathbf{R}$ the tensor rank.

Winograd 1971 [10] showed $\mathbf{R}(\langle 2, 2, 2\rangle) \geq 7$ over $\mathbb{Q}$.

Hopcroft and Kerr 1971 [6] showed $\mathbf{R}(\langle 2, 2, n\rangle) \geq \lceil 7n/2 \rceil$ over $\mathbb{F}_2$ and $\mathbf{R}(\langle 2, 3, 3\rangle) \geq 15$ over $\mathbb{F}_2$. They also mentioned $\mathbf{R}(\langle 2, 3, 4\rangle) \geq 19$ over $\mathbb{F}_2$, but without a formal proof.

Bläser 1999 [2] showed $\mathbf{R}(\langle l, m, n\rangle) \geq lm + mn + l - m + n - 3$ for $n \geq l \geq 2$ over algebraically closed fields. It implies the same lower bound over $\mathbb{Q}$ and $\mathbb{F}_2$.

Bläser 1999 [1] showed $\mathbf{R}(\langle n, n, n\rangle) \geq \frac{5}{2}n^2 - 3n$ over an arbitrary field.

Bläser 2003 [3] showed $\mathbf{R}(\langle n, m, n\rangle) \geq 2mn + 2n - m - 2$ for $m \geq n \geq 3$ over an arbitrary field.

2 Our Results

We prove that the tensor rank of the matrix multiplication tensor for two 3×3 matrices is at least 20 over $\mathbb{F}_2$. That is, any bilinear algorithm for multiplying two 3×3 matrices over $\mathbb{F}_2$ must use at least 20 scalar multiplications.

Theorem 1. $\mathbf{R}(\langle 3,3,3 \rangle) \geq 20$ *over* $\mathbb{F}_2$.

This improves the previous lower bound of 19 by Bläser 2003 [3].

We also complete the proof of $\mathbf{R}(\langle 2,3,4 \rangle) \geq 19$, which was skipped in Hopcroft and Kerr 1971 [6].

Theorem 2. $\mathbf{R}(\langle 2,3,4 \rangle) \geq 19$ *over* $\mathbb{F}_2$.

We further obtain the following lower bound for multiplying a 3×3 matrix by a 3×4 matrix over $\mathbb{F}_2$.

Theorem 3. $\mathbf{R}(\langle 3,3,4 \rangle) \geq 25$ *over* $\mathbb{F}_2$.

The table below summarizes the tensor rank lower and upper bounds over $\mathbb{F}_2$.

Matrix Shape	Previous LB	Our LB	UB
$2 \times 2 \times 2$	7 [10, 6]	7	7 [9]
$2 \times 2 \times 3$	11 [6]	11	11 ($\langle 2 \times 2 \times 2 : 7 \rangle + \langle 2 \times 2 \times 1 : 4 \rangle$)
$2 \times 2 \times 4$	14 [6]	14	14 ($\langle 2 \times 2 \times 2 : 7 \rangle + \langle 2 \times 2 \times 2 : 7 \rangle$)
$2 \times 3 \times 3$	15 [6]	15	15 [6]
$2 \times 3 \times 4$	18 [2], 19? [6]	$\boxed{19}$	20 [6]
$3 \times 3 \times 3$	19 [3]	$\boxed{20}$	23 [7]
$3 \times 3 \times 4$	24 [3]	$\boxed{25}$	29 [8]

The whole paper talks about the $\mathbb{F}_2$ case. The same techniques can be applied to other finite fields, but the running time will be much longer.

3 Preliminaries

3.1 Symmetries

The symmetries of the matrix multiplication tensor have been well studied [5, 4].

Consider the matrix multiplication problem $AB = C^T$, where A is an $l \times m$ matrix, B is an $m \times n$ matrix, and C is an $n \times l$ matrix. The matrix multiplication tensor is

$$\sum_{i=0}^{l-1} \sum_{j=0}^{m-1} \sum_{k=0}^{n-1} a_{i,j} \otimes b_{j,k} \otimes c_{k,i}.$$

For any invertible matrices P, Q, and R, $(PAQ^{-1})(QBR^{-1}) = (RCP^{-1})^T$. The matrix multiplication tensor rank is preserved. We call this the *sandwich symmetry*.

The next symmetry is the *cyclic symmetry*:

$$\mathbf{R}\left(\sum_{i=0}^{l-1}\sum_{j=0}^{m-1}\sum_{k=0}^{n-1} a_{i,j}\otimes b_{j,k}\otimes c_{k,i}\right) = \mathbf{R}\left(\sum_{i=0}^{l-1}\sum_{j=0}^{m-1}\sum_{k=0}^{n-1} b_{j,k}\otimes c_{k,i}\otimes a_{i,j}\right).$$

For square matrices $l = m = n$, we can transpose the three matrices but swap two of them. $B^T A^T = C^T$. Together with the cyclic symmetry, we have

$$\mathbf{R}\left(\sum_{i=0}^{n-1}\sum_{j=0}^{n-1}\sum_{k=0}^{n-1} a_{i,j}\otimes b_{j,k}\otimes c_{k,i}\right) = \mathbf{R}\left(\sum_{i=0}^{n-1}\sum_{j=0}^{n-1}\sum_{k=0}^{n-1} a_{j,i}\otimes c_{i,k}\otimes b_{k,j}\right).$$

We call it the *transpose symmetry*.

3.2 Restrictions

We consider the matrix multiplication problem with linear restrictions on matrix A. For example, the constraints $a_{i,j} = a_{j,i}$ give a symmetric A, and the constraints $a_{0,j} = 0$ reduce it to an $\langle l-1, m, n\rangle$ problem.

Some sets of restrictions are equivalent: they define the same subspace, or they lie in the same *orbit* under the symmetries (sandwich, cyclic, transpose). For example, $\{a_{0,0} = 0,\ a_{0,1} = 0\}$ and $\{a_{0,0} = 0,\ a_{0,0} + a_{0,1} = 0\}$ are equivalent because they define the same subspace. The constraints $\{a_{0,j} = 0 : 0 \le j < m\}$ and $\{a_{1,j} = 0 : 0 \le j < m\}$ are in the same orbit under the sandwich symmetry, since we can permute the rows of A by P.

Next, we aim to enumerate all distinct orbits of restrictions on A over $\mathbb{F}_2$. For each orbit, we select a canonical representative.

We can determine whether two restriction sets lie in the same subspace by performing Gauss-Jordan elimination into reduced row echelon form (RREF). RREF is also the lexicographically minimal form (possibly after reversing the order of rows or columns, depending on the chosen lexicographic order).

Algorithm 1 Enumerate restriction orbits on $l \times m$ matrix A over $\mathbb{F}_2$

Ensure: Returns *orbits*, a lexicographically ordered list of canonical representatives. A canonical representative is a lexicographically ordered list of restrictions. Each restriction is a $l \times m$ matrix over $\mathbb{F}_2$, or a binary integer of lm bits.

```
orbits ← empty list
ENUMERATE(empty list)
return orbits

function ENUMERATE(restrictions)
    if |restrictions| = lm then
        return
    end if
    last ← last element of restrictions (or 0 if empty)
    for restriction = last + 1 to 2^(lm) − 1 do
        if restriction is in subspace of restrictions then
            continue
        end if
        append restriction to restrictions
        if ISCANONICAL(restrictions) then
            append restrictions to orbits
            ENUMERATE(restrictions)
        end if
        remove last element from restrictions
    end for
end function

function ISCANONICAL(restrictions)
    for each full-rank matrix P and Q^(−1) in random order do
        for transpose in {false, true} if l = m = n do
            new_restrictions ← RREF of {PMQ^(−1) : M ∈ restrictions} (transpose M if transpose)
            if new_restrictions <_lex restrictions then
                return false
            end if
        end for
    end for
    return true
end function
```

Algorithm 1 finds 10 orbits in the $\langle 2,2,2\rangle$ case and 496 orbits in the $\langle 3,3,3\rangle$ case within a minute.

4 A Toy Example: $\mathbf{R}(\langle 2,2,2\rangle) \geq 7$ over $\mathbb{F}_2$

We take $\mathbf{R}(\langle 2,2,2\rangle)$ as an example to show how the tensor rank lower bound techniques work.

Algorithm 1 found 10 orbits of restrictions on A over $\mathbb{F}_2$ as below.

Orbit 0: $\{a_{0,0} = 0,\ a_{0,1} = 0,\ a_{1,0} = 0,\ a_{1,1} = 0\}$

$A = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$. The matrix multiplication tensor is $T_0 = 0$. $\mathbf{R}(T_0) = 0$.

Orbit 1: $\{a_{0,0} = 0,\ a_{0,1} = 0,\ a_{1,0} = 0\}$

$A = \begin{pmatrix} 0 & 0 \\ 0 & a_{1,1} \end{pmatrix}$. The matrix multiplication tensor is $T_1 = a_{1,1} \otimes b_{1,0} \otimes c_{0,1} + a_{1,1} \otimes b_{1,1} \otimes c_{1,1}$.

Lemma 1 (Flattening)**.** *The rank of a trilinear tensor is greater than or equal to the matrix rank after flattening two dimensions.*

In this case, we flatten a and b into one dimension x, and get $x_{1,1,1,0} \otimes c_{0,1} + x_{1,1,1,1} \otimes c_{1,1}$. It has matrix rank 2. So, $\mathbf{R}(T_1) \geq 2$.

Orbit 2: $\{a_{0,0} = 0,\ a_{0,1} + a_{1,0} = 0,\ a_{1,1} = 0\}$

$A = \begin{pmatrix} 0 & a_{0,1} \\ a_{0,1} & 0 \end{pmatrix}$. The matrix multiplication tensor is $T_2 = a_{0,1} \otimes b_{0,0} \otimes c_{0,1} + a_{0,1} \otimes b_{0,1} \otimes c_{1,1} + a_{0,1} \otimes b_{1,0} \otimes c_{0,0} + a_{0,1} \otimes b_{1,1} \otimes c_{1,0}$. By Flattening Lemma (Lemma 1), we flatten a and b into one dimension x, and get $x_{0,1,0,0} \otimes c_{0,1} + x_{0,1,0,1} \otimes c_{1,1} + x_{0,1,1,0} \otimes c_{0,0} + x_{0,1,1,1} \otimes c_{1,0}$. It has matrix rank 4. So, $\mathbf{R}(T_2) \geq 4$.

Orbit 3: $\{a_{0,0} = 0,\ a_{0,1} = 0\}$

$A = \begin{pmatrix} 0 & 0 \\ a_{1,0} & a_{1,1} \end{pmatrix}$. The matrix multiplication tensor is $T_3 = a_{1,0} \otimes b_{0,0} \otimes c_{0,1} + a_{1,0} \otimes b_{0,1} \otimes c_{1,1} + a_{1,1} \otimes b_{1,0} \otimes c_{0,1} + a_{1,1} \otimes b_{1,1} \otimes c_{1,1}$. By Flattening Lemma (Lemma 1), we flatten a and b into one dimension x, and get $x_{1,0,0,0} \otimes c_{0,1} + x_{1,0,0,1} \otimes c_{1,1} + x_{1,1,1,0} \otimes c_{0,1} + x_{1,1,1,1} \otimes c_{1,1}$. It has matrix rank 4. So, $\mathbf{R}(T_3) \geq 4$.

Orbit 4: $\{a_{0,0} = 0,\ a_{0,1} + a_{1,0} = 0\}$

$A = \begin{pmatrix} 0 & a_{0,1} \\ a_{0,1} & a_{1,1} \end{pmatrix}$. The matrix multiplication tensor is $T_4 = a_{0,1} \otimes b_{0,0} \otimes c_{0,1} + a_{0,1} \otimes b_{0,1} \otimes c_{1,1} + a_{0,1} \otimes b_{1,0} \otimes c_{0,0} + a_{0,1} \otimes b_{1,1} \otimes c_{1,0} + a_{1,1} \otimes b_{1,0} \otimes c_{0,1} + a_{1,1} \otimes b_{1,1} \otimes c_{1,1}$.

For any orbit, the Flattening Lemma (Lemma 1) can only give a lower bound at most 4. In this orbit, we need another technique.

Lemma 2 (Lemma 2 in [6])**.** *Let $F = \{f_0, f_1, \ldots, f_{s-1}, \ldots, f_{t-1}\}$ be a set of expressions, where $f_0, f_1, \ldots, f_{s-1}$ are independent, and each can be expressed as a single product. If F can be computed with p multiplications, then there exists an algorithm for F with p multiplications in which s of the multiplications are $f_0, f_1, \ldots, f_{s-1}$.*

Choose $F = \{a_{0,1} \otimes b_{1,0}, a_{0,1} \otimes b_{1,1}\}$. $a_{0,1} \otimes b_{1,0}$ and $a_{0,1} \otimes b_{1,1}$ are independent, and single products. By Lemma 2, we can assume the decomposition scheme contains $a_{0,1} \otimes b_{1,0} \otimes (c_{0,0} + [c_{0,1}] + [c_{1,1}]) +$ and $a_{0,1} \otimes b_{1,1} \otimes (c_{1,0} + [c_{0,1}] + [c_{1,1}])$, where $[x]$ means x may or may not be present. After removing the two terms from the decomposition scheme, the remaining tensor is $T_4 + a_{0,1} \otimes b_{1,0} \otimes (c_{0,0} + [c_{0,1}] + [c_{1,1}]) + a_{0,1} \otimes b_{1,1} \otimes (c_{1,0} + [c_{0,1}] + [c_{1,1}])$. By enumerating all the $2^4 = 16$ cases of the presence of the 4 square-bracket terms, we find that all of them have rank 4 by the Flattening Lemma (Lemma 1). So, $\mathbf{R}(T_4) \geq 4 + 2 = 6$.

Orbit 5: $\{a_{0,0} = 0,\ a_{1,1} = 0\}$

$A = \begin{pmatrix} 0 & a_{0,1} \\ a_{1,0} & 0 \end{pmatrix}$. The matrix multiplication tensor is $T_5 = a_{0,1} \otimes b_{1,0} \otimes c_{0,0} + a_{0,1} \otimes b_{1,1} \otimes c_{1,0} + a_{1,0} \otimes b_{0,0} \otimes c_{0,1} + a_{1,0} \otimes b_{0,1} \otimes c_{1,1}$. By Flattening Lemma (Lemma 1), we flatten a and b into one dimension x, and get $x_{0,1,1,0} \otimes c_{0,0} + x_{0,1,1,1} \otimes c_{1,0} + x_{1,0,0,0} \otimes c_{0,1} + x_{1,0,0,1} \otimes c_{1,1}$. It has matrix rank 4. So, $\mathbf{R}(T_5) \geq 4$.

Orbit 6: $\{a_{0,1} + a_{1,0} = 0,\ a_{0,0} + a_{0,1} + a_{1,1} = 0\}$

$A = \begin{pmatrix} a_{0,0} & a_{0,1} \\ a_{0,1} & a_{0,0} + a_{0,1} \end{pmatrix}$. The matrix multiplication tensor is $T_6 = a_{0,0} \otimes b_{0,0} \otimes c_{0,0} + a_{0,0} \otimes b_{0,1} \otimes c_{1,0} + a_{0,0} \otimes b_{1,0} \otimes c_{0,1} + a_{0,0} \otimes b_{1,1} \otimes c_{1,1} + a_{0,1} \otimes b_{0,0} \otimes c_{0,1} + a_{0,1} \otimes b_{0,1} \otimes c_{1,1} + a_{0,1} \otimes b_{1,0} \otimes c_{0,0} + a_{0,1} \otimes b_{1,0} \otimes c_{0,1} + a_{0,1} \otimes b_{1,1} \otimes c_{1,0} + a_{0,1} \otimes b_{1,1} \otimes c_{1,1}$.

First, we have $\mathbf{R}(T_6) \geq 4$ by flattening the tensor. Then, we apply the substitution method on A.

If $a_{0,0}$ appears at least twice in the decomposition scheme, we set $a_{0,0} = 0$. Then, $A = \begin{pmatrix} 0 & a_{0,1} \\ a_{0,1} & 0 \end{pmatrix}$. It is in Orbit 5. So, $\mathbf{R}(T_6) \geq \mathbf{R}(T_5) + 2 \geq 4 + 2 = 6$.

If $a_{0,1}$ appears at least twice in the decomposition scheme, we set $a_{0,1} = 0$. Then, $A = \begin{pmatrix} a_{0,0} & 0 \\ 0 & a_{0,0} \end{pmatrix}$. It is in Orbit 5 by swapping two rows and two columns of A. So, $\mathbf{R}(T_6) \geq \mathbf{R}(T_5) + 2 \geq 4 + 2 = 6$.

If $a_{0,0} + a_{0,1}$ appears at least twice in the decomposition scheme, we set $a_{0,0} + a_{0,1} = 0$. Then, $A = \begin{pmatrix} a_{0,0} & a_{0,0} \\ a_{0,0} & 0 \end{pmatrix}$. It is in Orbit 5 by adding the second row to the first row. So, $\mathbf{R}(T_6) \geq \mathbf{R}(T_5) + 2 \geq 4 + 2 = 6$.

Because $\mathbf{R}(T_6) \geq 4$, at least one of the three cases must hold. Hence $\mathbf{R}(T_6) \geq 6$.

Orbit 7: $\{a_{0,0} = 0\}$

$A = \begin{pmatrix} 0 & a_{0,1} \\ a_{1,0} & a_{1,1} \end{pmatrix}$. We can reduce it to Orbit 4 by adding the restriction $a_{0,1} + a_{1,0} = 0$. So, $\mathbf{R}(T_7) \geq \mathbf{R}(T_4) \geq 6$.

Orbit 8: $\{a_{0,1} + a_{1,0} = 0\}$

$A = \begin{pmatrix} a_{0,0} & a_{0,1} \\ a_{0,1} & a_{1,1} \end{pmatrix}$. We can reduce it to Orbit 4 by adding a restriction $a_{0,0} = 0$. So, $\mathbf{R}(T_8) \geq \mathbf{R}(T_4) \geq 6$.

Orbit 9: $\{\}$

$A = \begin{pmatrix} a_{0,0} & a_{0,1} \\ a_{0,1} & a_{1,1} \end{pmatrix}$. This is the general 2×2 case. $T_9 = \sum_{i=0}^{1} \sum_{j=0}^{1} \sum_{k=0}^{1} a_{i,j} \otimes b_{j,k} \otimes c_{k,i}$.

First, it is easy to show that $\mathbf{R}(T_9) \geq 1$. Then, we apply the substitution method on A.

If $a_{0,0}$ appears at least once in the decomposition scheme, we set $a_{0,0} = 0$. Then, we reduce it to Orbit 7. So, $\mathbf{R}(T_9) \geq \mathbf{R}(T_7) + 1 \geq 6 + 1 = 7$. The same argument applies to $a_{0,1}$, $a_{1,0}$, $a_{1,1}$, $a_{0,0} + a_{0,1}$, $a_{1,0} + a_{1,1}$, $a_{0,0} + a_{1,0}$, $a_{0,1} + a_{1,1}$, and $a_{0,0} + a_{0,1} + a_{1,0} + a_{1,1}$ by the sandwich symmetry.

If $a_{0,1} + a_{1,0}$ appears at least once in the decomposition scheme, we set $a_{0,1} + a_{1,0} = 0$. Then, we reduce it to Orbit 8. So, $\mathbf{R}(T_9) \geq \mathbf{R}(T_8) + 1 \geq 6 + 1 = 7$. The same argument applies to $a_{0,0} + a_{1,1}$, $a_{0,0} + a_{0,1} + a_{1,0}$, $a_{0,0} + a_{0,1} + a_{1,1}$, $a_{0,0} + a_{1,0} + a_{1,1}$, $a_{0,1} + a_{1,0} + a_{1,1}$ by the sandwich symmetry.

We list all the possible restrictions on A. So, $\mathbf{R}(T_9) \geq 7$.

5 Rank Lower Bound Techniques

In this section, we formally describe the four techniques used in the toy example.

5.1 Flattening

Flatten dimensions a and b, flatten dimensions b and c, and flatten dimensions c and a. Then, take the maximum of the three matrix ranks.

5.2 Forced Product

Example: Orbit 4.

Group the tensor by c into t terms. Example: $T_4 = (a_{0,1} \otimes b_{0,0} + a_{1,1} \otimes b_{1,0}) \otimes c_{0,1} + (a_{0,1} \otimes b_{0,1} + a_{1,1} \otimes b_{1,1}) \otimes c_{1,1} + a_{0,1} \otimes b_{1,0} \otimes c_{0,0} + a_{0,1} \otimes b_{1,1} \otimes c_{1,0}$. $t = 4$

Pick the terms that have matrix rank 1 on a and b, i.e., those that can be expressed as a single product. For example, the first term $a_{0,1} \otimes b_{0,0} + a_{1,1} \otimes b_{1,0}$ has rank 2; the second term $a_{0,1} \otimes b_{0,1} + a_{1,1} \otimes b_{1,1}$ has rank 2; the third term $a_{0,1} \otimes b_{1,0}$ has rank 1; and the fourth term $a_{0,1} \otimes b_{1,1}$ has rank 1. So, we pick the third and fourth terms: $a_{0,1} \otimes b_{1,0}$ and $a_{0,1} \otimes b_{1,1}$.

Greedily choose the largest linearly independent subset of the picked terms of rank 1. Denote the subset size by s. In this example, the two terms are linearly independent, and we choose both.

Apply Lemma 2 by setting $f_0, f_1, \ldots, f_{s-1}$ to the subset of size s we just chose. They are independent, and each can be expressed as a single product. So, we can assume the decomposition scheme contains the s terms. After removing the s terms from the decomposition scheme, we compute the rank of the remaining tensor. There are $s(t-s)$ terms which may or may not be present. We enumerate all the $2^{s(t-s)}$ cases of the presence of the $s(t-s)$ terms, and compute the rank lower bound for each case by Flattening Technique, and take the minimum. In practice, if $s(t-s) \geq 32$, we skip this technique because it runs too slowly.

Finally, we repeat the whole process on the three cyclic permutations of the tensor, and take the maximum of the three ranks.

5.3 Degenerate Reduction

Example: Orbit 7.

If one restriction set X is contained in another restriction set Y, then the tensor rank of T_X is at least the tensor rank of T_Y.

5.4 Backtracking

For Orbit 6, the rank lower bound proof was constructed by hand and is concise. In this section, we present an automated approach to generating the proof, which, while longer, is found systematically.

First, we use the previous three techniques to obtain a rank lower bound, denoted by *known_lower_bound*. Then we enumerate all possible factors of A, up to *known_lower_bound* factors. In each case we set a subset of the factors to zero and reduce to another orbit.

In the example of Orbit 6, there are $2^4 - 1$ possible non-zero factors of A: $[a_{0,0}] + [a_{0,1}] + [a_{1,0}] + [a_{1,1}]$ except for 0. Given the restriction set $\{a_{0,1} + a_{1,0} = 0,\ a_{0,0} + a_{0,1} + a_{1,1} = 0\}$, many factors are

redundant. For example, $a_{0,1}$ and $a_{1,0}$ are equal because $a_{0,1} + a_{1,0} = 0$. We pick a canonical factor (the lexicographically minimum one) among each set of equal factors. The minimum factor should not contain $a_{i,j}$ if (i, j) is the most significant bit of a restriction in *restriction_set*. In the example, $(1, 0)$ is the most significant bit of the restriction $a_{0,1} + a_{1,0} = 0$, and $(1, 1)$ is the most significant bit of the restriction $a_{0,0} + a_{0,1} + a_{1,1} = 0$. So, the canonical factor should not contain $a_{1,0}$ or $a_{1,1}$. It is sum of a non-empty subset of the others: $\{a_{0,0},\ a_{0,1}\}$, i.e. the canonical factors are $a_{0,0}$, $a_{0,1}$, and $a_{0,0} + a_{0,1}$.

Algorithm 2 Canonical Factors Enumeration

Require: *restriction_set*, a set of restrictions on A, in RREF form.
Ensure: Returns *canonical_factors*, a list of canonical factors on A.

```
canonical_factors ← empty list
for factor = 1 to 2^lm − 1 do
    for restriction ∈ restriction_set do
        if factor xor restriction < factor then
            continue factor-for-loop                    ▷ The factor is not canonical
        end if
    end for
    append factor to canonical_factors
end for
```

We have $\mathbf{R}(T_6) \geq 4$ by the Flattening Technique. Then, we enumerate factors in $\{a_{0,0}, a_{0,1}, a_{0,0} + a_{0,1}\}^4$. As the factors are unordered, we assume the factors are non-decreasing.

We are listing the running progress of the backtracking algorithm.

$a_{0,0}$: We do not have a proof for $\mathbf{R}(T_6) \geq 6$. Go deeper.

$a_{0,0}$, $a_{0,0}$: Set $a_{0,0} = 0$. Then, we reduce it to Orbit 5. So, $\mathbf{R}(T_6) \geq \mathbf{R}(T_5) + 2 \geq 4 + 2 = 6$. Proved. We do not need to go deeper.

$a_{0,0}$, $a_{0,1}$: We do not have a proof for $\mathbf{R}(T_6) \geq 6$. Go deeper.

$a_{0,0}$, $a_{0,1}$, $a_{0,1}$: Set $a_{0,1} = 0$. Then, we reduce it to Orbit 5. So, $\mathbf{R}(T_6) \geq \mathbf{R}(T_5) + 2 \geq 4 + 2 = 6$. Proved. We do not need to go deeper.

$a_{0,0}$, $a_{0,1}$, $a_{0,0} + a_{0,1}$: We do not have a proof for $\mathbf{R}(T_6) \geq 6$. Go deeper.

$a_{0,0}$, $a_{0,1}$, $a_{0,0} + a_{0,1}$, $a_{0,0} + a_{0,1}$: Set $a_{0,0} + a_{0,1} = 0$. Then, we reduce it to Orbit 5. So, $\mathbf{R}(T_6) \geq \mathbf{R}(T_5) + 2 \geq 4 + 2 = 6$. Proved. We do not need to go deeper.

$a_{0,0}$, $a_{0,0} + a_{0,1}$: We do not have a proof for $\mathbf{R}(T_6) \geq 6$. Go deeper. $a_{0,0}$, $a_{0,0} + a_{0,1}$, $a_{0,0} + a_{0,1}$: Set $a_{0,0} + a_{0,1} = 0$. Then, we reduce it to Orbit 5. So, $\mathbf{R}(T_6) \geq \mathbf{R}(T_5) + 2 \geq 4 + 2 = 6$. Proved. We do not need to go deeper.

$a_{0,1}$: We do not have a proof for $\mathbf{R}(T_6) \geq 6$. Go deeper.

$a_{0,1}$, $a_{0,1}$: Set $a_{0,1} = 0$. Then, we reduce it to Orbit 5. So, $\mathbf{R}(T_6) \geq \mathbf{R}(T_5) + 2 \geq 4 + 2 = 6$. Proved. We do not need to go deeper.

$a_{0,1}$, $a_{0,0} + a_{0,1}$: We do not have a proof for $\mathbf{R}(T_6) \geq 6$. Go deeper.

$a_{0,1}$, $a_{0,0} + a_{0,1}$, $a_{0,0} + a_{0,1}$: Set $a_{0,0} + a_{0,1} = 0$. Then, we reduce it to Orbit 5. So, $\mathbf{R}(T_6) \geq \mathbf{R}(T_5) + 2 \geq 4 + 2 = 6$. Proved. We do not need to go deeper.

$a_{0,0} + a_{0,1}$: We do not have a proof for $\mathbf{R}(T_6) \geq 6$. Go deeper.

$a_{0,0} + a_{0,1}$, $a_{0,0} + a_{0,1}$: Set $a_{0,0} + a_{0,1} = 0$. Then, we reduce it to Orbit 5. So, $\mathbf{R}(T_6) \geq \mathbf{R}(T_5) + 2 \geq 4 + 2 = 6$. Proved. We do not need to go deeper.

Formally, we have the following algorithm:

Algorithm 3 Backtracking Algorithm

Require: *restriction_set*, a set of restrictions on A, in RREF form.
Require: the best-known rank lower bound of each orbit.
Require: the target lower bound
Ensure: Returns the rank lower bound with a *proof*

```
Compute canonical_factors
function BACKTRACK(factors)
    if factors is not empty then
        for each sub-list sub_factors of factors containing the last factor do
            extended_restrictions ← restriction_set + sub_factors
            Find the orbit of extended_restrictions, by enumerating all the possible symmetries
followed by Gauss-Jordan elimination
            if |sub_factors| + best-known rank lower bound of orbit of extended_restrictions ≥
the target lower bound then
                Save |sub_factors| and the symmetry to proof
                return True
            end if
        end for
    end if
    last_factor ← the last element of factors, or 2^lm − 1 if factors is empty
    for factor in canonical_factors and factor ≤ last_factor do
        if BACKTRACK(factors + [factor]) is False then
            return False
        end if
    end for
    return True
end function
```

This algorithm works for $\langle 2, 2, 2 \rangle$, but it is too slow for larger matrices. We need to optimize it.

5.4.1 Optimization: Multi-threading

We parallelize all the orbits with the same number of restrictions, because the backtracking for an orbit depends on the other orbits with more restrictions. After processing all orbits with a given number of restrictions, we proceed to orbits with one fewer restriction, until we reach the orbits with no restrictions (the general case).

Also, we parallelize the *factor* for-loop in the first layer of the backtracking function.

5.4.2 Optimization: Restriction Map

Line 6 in Algorithm 3 needs to enumerate all the $l \times l$ full-rank matrices P, all the $m \times m$ full-rank matrices Q^{-1}, and whether to transpose. It is too slow. To get a trade-off between speed and memory usage, we can use a hash map that stores, for each orbit, the pairs $(Q^{-1}, \mathit{transpose})$. When looking up, we only enumerate over P. This makes the lookup time roughly a square root faster.

To improve the inserting speed, we use sharding. We split the hash map into multiple shards (e.g. 997 shards), each one has a mutex lock, and insert the elements in parallel.

5.4.3 Optimization: Local Map and Memory Management

The restriction map is faster, but it still needs to enumerate all the Ps. So, we build a local cache in each thread for each orbit: a hash map from *sub_factors* to the lower bound for that sub-list. In some cases, the local cache still grows too large. So, when the local cache is getting larger than some threshold, we randomly delete half of the elements to avoid OOM. In practice, the threshold is set to 3×10^6.

5.4.4 Optimization: Incremental Target Lower Bound

We set the target lower bound to the best-known rank lower bound plus one. So, it improves the rank lower bound one at a time.

5.4.5 Optimization: Step Limit

In some cases, the backtracking algorithm runs too slowly. So, we limit the number of steps. When it reaches the limit, we cancel the search.

6 Proof and Verification

For each orbit we record which technique was used in the proof.

If the Forced Product technique is used, we save the cyclic permutation type (a number in $\{0, 1, 2\}$) in the proof.

If the Degenerate Reduction technique is used, we save the extra restriction $Y \setminus X$ in the proof.

If the Backtracking technique is used, the proof can be very long. To achieve a trade-off between proof size and verification cost, we save the following at Line 9 in Algorithm 3:

- the size of *factors*;
- the index set of *sub_factors* within *factors*;
- the symmetry (the matrices P, Q^{-1}, and whether to transpose).

7 Applications

7.1 $\mathbf{R}(\langle 3, 3, 3\rangle) \geq 20$ over $\mathbb{F}_2$

With step limit 10^7, our program takes 71 minutes to find the proof on a MacBook Air M4 with 8 cores and 16 GB memory.

The proof is about 32 MiB. It can be verified on the same computer within 10 seconds.

7.2 $\mathbf{R}(\langle 2, 3, 4\rangle) \geq 19$ over $\mathbb{F}_2$

Our program actually proves $\mathbf{R}(\langle 3, 2, 4\rangle) \geq 19$ over $\mathbb{F}_2$. The step limit is 10^7. It takes 110 minutes to find the proof on a MacBook Air M4 with 8 cores and 16 GB memory.

The proof is under 100 KiB and can be verified on the same computer in 110 minutes. Although verification takes as long as the search, it remains useful: the verifier program is shorter than the search program and is therefore easier to audit for correctness.

7.3 $\mathbf{R}(\langle 3, 3, 4\rangle) \geq 25$ over $\mathbb{F}_2$

With step limit 10^8, our program takes 3.5 days to find the proof on the cloud server with 64 CPU cores and 128 GB memory. It costs about 168 USD.

The proof is about 626 MiB. It can be verified on the same computer within 3.5 hours, which costs about 7 USD.

7.4 Implementation

The code and the proof files are available at
`https://github.com/wcgbg/matrix-multiplication-lower-bound-n3r20f2`.

References

[1] Markus Bläser. A $5/2n^2$-lower bound for the multiplicative complexity of $n \times n$-matrix multiplication. In *Proceedings of the 40th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 45–50. IEEE, 1999.

[2] Markus Bläser. Lower bounds for the multiplicative complexity of matrix multiplication. *computational complexity*, 8(3):203–226, 1999.

[3] Markus Bläser. On the complexity of the multiplication of matrices of small formats. *Journal of Complexity*, 19(1):43–60, 2003.

[4] Luca Chiantini, Christian Ikenmeyer, Joseph M Landsberg, and Giorgio Ottaviani. The geometry of rank decompositions of matrix multiplication i: 2×2 matrices. *Experimental Mathematics*, 28(3):322–327, 2019.

[5] Hans F de Groote. On varieties of optimal algorithms for the computation of bilinear mappings i. the isotropy group of a bilinear mapping. *Theoretical Computer Science*, 7(1):1–24, 1978.

[6] John E Hopcroft and Leslie R Kerr. On minimizing the number of multiplications necessary for matrix multiplication. *SIAM Journal on Applied Mathematics*, 20(1):30–36, 1971.

[7] Julian David Laderman. A noncommutative algorithm for multiplying 3×3 matrices using 23 multiplications. *Bulletin of the American Mathematical Society*, 82(1):126–128, January 1976.

[8] Alexey V Smirnov. The bilinear complexity and practical algorithms for matrix multiplication. *Computational Mathematics and Mathematical Physics*, 53(12):1781–1795, 2013.

[9] Volker Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4):354–356, August 1969.

[10] Shmuel Winograd. On multiplication of 2×2 matrices. *Linear algebra and its applications*, 1971.